

مستندات فایل polyp_transforms.py

تبدیلات داده برای پروژه سگمنتیشن پولیپ

پایان نامه کارشناسی ارشد - مهندسی کامپیوتر، گرایش هوش مصنوعی

۷ مهر ۱۴۰۵

فهرست مطالب

۲	۱	مقدمه
۲	۲	کلاس ResizeWithRatio
۲	۱.۲	هدف کلاس
۲	۲.۲	سازنده‌ی کلاس (<code>__init__</code>)
۳	۳.۲	متد <code>__call__</code>
۳	۱.۳.۲	تحلیل مرحله به مرحله
۴	۴.۲	جریان ابعاد در <code>ResizeWithRatio</code>
۴	۳	تابع get_polyp_transforms
۴	۱.۳	هدف تابع
۵	۲.۳	Pipeline آموزش (<code>train_transforms</code>)
۶	۳.۳	Pipeline اعتبارسنجی/تست (<code>val_test_transforms</code>)
۸	۴	تحلیل اثر هر Transform روی داده
۸	۱.۴	گام ۱: <code>LoadImageD</code>
۸	۲.۴	گام ۲: <code>EnsureChannelFirstD</code>
۸	۳.۴	گام ۳: <code>ScaleIntensityRanged</code>
۸	۴.۴	گام ۴: <code>NormalizeIntensityD</code>
۹	۵.۴	گام ۵: <code>ResizeWithRatio</code>
۹	۶.۴	گام ۶: <code>ResizeWithPadOrCrop</code>
۹	۷.۴	گام‌های ۷-۹: <code>Augmentation</code>
۱۰	۸.۴	گام ۱۰: <code>EnsureTyped</code>
۱۰	۵	جریان کامل داده - نمودار
۱۰	۶	نسخه‌های بهبوددهنده (برای آینده)
۱۰	۱.۶	بهبود ۱: <code>RandRotateD</code> (جایگزین <code>RandRotate90D</code>)
۱۱	۲.۶	بهبود ۲: <code>RandZoomD</code>
۱۱	۳.۶	بهبود ۳: <code>RandAdjustContrastD</code>
۱۱	۴.۶	بهبود ۴: <code>RandGaussianNoiseD</code>
۱۱	۵.۶	بهبود ۵: <code>RandShiftIntensityD</code>
۱۱	۶.۶	بهبود ۶: <code>RandGaussianSmoothD</code>
۱۱	۷	جمع بندی

۱ مقدمه

فایل `polyp_transforms.py` مسئول تعریف تبدیلات داده (Data Transforms) برای پروژه سگمنتیشن پولیپ است. این تبدیلات، تصاویر RGB کولونوسکوپی و ماسک‌های متناظرشان را از حالت خام (فایل‌های PNG به‌های tensor آماده برای ورود به مدل SAM-based تبدیل می‌کنند. این فایل شامل دو بخش اصلی است:

۱. کلاس **ResizeWithRatio**: یک transform سفارشی که تصویر و ماسک را با حفظ نسبت ابعاد تغییر اندازه می‌دهد.

۲. تابع **get_polyp_transforms**: تابعی که دو pipeline کامل (آموزش و ارزیابی) می‌سازد.

جریان کلی داده: `file` (PNG) → `LoadImageD` → `EnsureChannelFirstD`
→ `ScaleIntensityRangeD` → `NormalizeIntensityD` → `ResizeWithRatio` → `EnsureTyped` → `Augmentation` → `ResizeWithPadOrCrop`

۲ کلاس **ResizeWithRatio**

۱.۲ هدف کلاس

کلاس `ResizeWithRatio` یک transform سفارشی است که دو کار مهم انجام می‌دهد:

۱. تغییر اندازه با حفظ نسبت ابعاد: به جای اینکه تصویر را به ابعاد مربع بکشد (که پولیپ دایره‌ای را به بیضی تبدیل می‌کند)، ضلع بزرگ‌تر را به `image_size` می‌رساند و ضلع دیگر را با همان نسبت مقیاس می‌دهد.

۲. تفکیک حالت درون‌یابی: تصویر با `bilinear` (نرم و پیوسته) و ماسک با `nearest` (حفظ مقادیر دودویی) تغییر اندازه می‌یابد.

۲.۲ سازنده‌ی کلاس (`__init__`)

```
1 def __init__(self, image_key, mask_key, image_size,
2             image_mode='bilinear', mask_mode='nearest'):
3     super().__init__(keys=[image_key, mask_key])
4     self.image_key = image_key
5     self.mask_key = mask_key
6     self.image_size = image_size
7     self.image_mode = image_mode
8     self.mask_mode = mask_mode
9     self._image_align_corners = (
10         None if image_mode in ('nearest', 'area') else False
11     )
```

توضیح پارامترها:

پارامتر	توضیح
<code>image_key</code>	کلید تصویر در دیکشنری داده (مثلاً "image")
<code>mask_key</code>	کلید ماسک در دیکشنری داده (مثلاً "mask")
<code>image_size</code>	اندازه‌ی هدف برای ضلع بزرگ‌تر (مثلاً ۱۰۲۴)
<code>image_mode</code>	حالت درون‌یابی برای تصویر (پیش‌فرض: <code>bilinear</code>)
<code>mask_mode</code>	حالت درون‌یابی برای ماسک (پیش‌فرض: <code>nearest</code>)
<code>_image_align_corners</code>	مقدار <code>align_corners</code> برای تصویر. اگر <code>mode</code> یکی از <code>nearest</code> یا <code>area</code> باشد، <code>None</code> ؛ در غیر این صورت <code>False</code> .

نکته: align_corners فقط برای های bilinear، bicubic و trilinear معنا دارد. برای nearest و area باید None باشد. این کلاس این تفکیک را به درستی انجام می‌دهد.

۳.۲ متد __call__

```

1 def __call__(self, data):
2     # بررسی وجود کلی داده‌ها : مرحله ۱
3     if self.image_key not in data:
4         raise KeyError(f"Image key '{self.image_key}' not found in data")
5     if self.mask_key not in data:
6         raise KeyError(f"Mask key '{self.mask_key}' not found in data")
7
8     # کپی سطحی دی‌کشنری : مرحله ۲
9     d = dict(data)
10    image = d[self.image_key]
11    mask = d[self.mask_key]
12
13    # (مجاز است 3D) بررسی ابعاد فقط : مرحله ۳
14    if image.ndim != 3:
15        raise ValueError(f"Expected 3D tensor (C, H, W) for image")
16    if mask.ndim != 3:
17        raise ValueError(f"Expected 3D tensor (C, H, W) for mask")
18
19    # محاسبه‌ی ابعاد جدید با حفظ نسبت : مرحله ۴
20    h, w = image.shape[-2], image.shape[-1]
21
22    if h == w:
23        new_h = new_w = self.image_size
24    elif h > w:
25        new_h = self.image_size
26        new_w = round(w * (self.image_size / h))
27    else:
28        new_w = self.image_size
29        new_h = round(h * (self.image_size / w))
30
31    # F.interpolate برای batch افزودن بُعد : مرحله ۵
32    img_4d = image.unsqueeze(0)
33    mask_4d = mask.unsqueeze(0)
34
35    # تغیری را اندازه‌ی تپو روماسک : مرحله ۶
36    resized_img = F.interpolate(
37        img_4d, size=(new_h, new_w),
38        mode=self.image_mode,
39        align_corners=self._image_align_corners
40    )
41    resized_mask = F.interpolate(
42        mask_4d, size=(new_h, new_w),
43        mode=self.mask_mode
44    )
45
46    # و ذخیره batch حذف بُعد : مرحله ۷
47    d[self.image_key] = resized_img.squeeze(0)
48    d[self.mask_key] = resized_mask.squeeze(0)
49
50    return d

```

۱.۳.۲ تحلیل مرحله به مرحله

مرحله ۱ — بررسی وجود کلیدها:

• اگر image_key یا mask_key در دیکشنری نباشد، خطای KeyError پرتاب می‌شود.

- این کار از خطاهای مبهم در مراحل بعدی جلوگیری می‌کند.

مرحله ۲ — کپی سطحی دیکشنری:

- `dict(data)` یک دیکشنری جدید می‌سازد.
- `tensor`های کپی نمی‌شوند (فقط مرجع کپی می‌شود) $\rightarrow$ کم‌هزینه.

مرحله ۳ — بررسی ابعاد:

- فقط `3D tensor`های (C, H, W) پذیرفته می‌شوند.
- اگر تصویر D^2 (بدون بُعد کانال) یا D^4 باشد، خطا داده می‌شود.

مرحله ۴ — محاسبه‌ی ابعاد با حفظ نسبت:

جدول ۱: محاسبه‌ی ابعاد جدید با حفظ نسبت ابعاد

ورودی (H, W)	شرط	new_h	new_w	خروجی
(529, 622)	$h < w$	$\text{round}(529 \times 1024/622) = 871$	1024	(871, 1024)
(622, 529)	$h > w$	1024	$\text{round}(529 \times 1024/622) = 871$	(1024, 871)
(512, 512)	$h == w$	1024	1024	(1024, 1024)
(1920, 1072)	$h > w$	1024	$\text{round}(1072 \times 1024/1920) = 572$	(1024, 572)

مرحله ۵ — افزودن بُعد batch:

- `F.interpolate` حداقل ۴ بُعد می‌خواهد: (N, C, H, W) .
- `unsqueeze(0)` بُعد batch را اضافه می‌کند.

مرحله ۶ — تغییر اندازه:

- تصویر: با `mode='bilinear'` و `align_corners=False`.
- ماسک: با `mode='nearest'` و بدون `align_corners`.

مرحله ۷ — حذف بُعد batch:

- `squeeze(0)` بُعد batch را حذف می‌کند.
- خروجی به شکل اولیه برمی‌گردد (فقط ابعاد spatial تغییر کرده).

۴.۲ جریان ابعاد در `ResizeWithRatio`

۳ تابع `get_polyp_transforms`

۱.۳ هدف تابع

این تابع دو pipeline کامل می‌سازد:

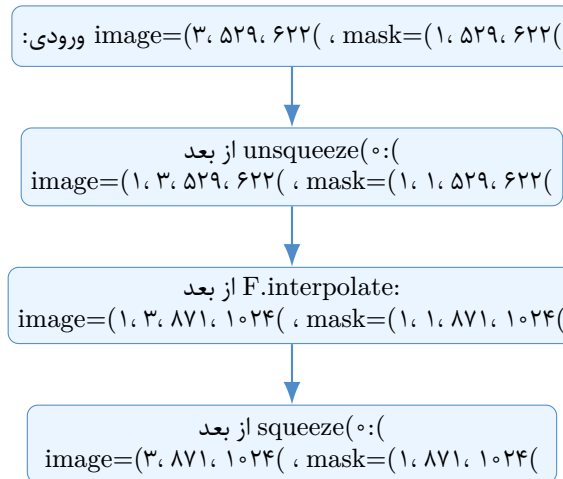
- **train_transforms**: برای آموزش (شامل `augmentation`).
- **val_test_transforms**: برای اعتبارسنجی و تست (بدون `augmentation`).

```

def get_polyp_transforms(args, sam_image_dim=None, get_bbox=False):
    KEYS = [args['image_key'], args['mask_key']]
    IMAGENET_MEAN = [0.485, 0.456, 0.406]
    IMAGENET_STD = [0.229, 0.224, 0.225]
    # ...

```

آماره‌های `ImageNet`



شکل ۱: جریان ابعاد در کلاس ResizeWithRatio

IMAGENET_STD	IMAGENET_MEAN	کانال
۲۲۹.۰	۴۸۵.۰	R
۲۲۴.۰	۴۵۶.۰	G
۲۲۵.۰	۴۰۶.۰	B

چرا این آماره‌ها؟ این مقادیر میانگین و انحراف معیار تصاویر ImageNet هستند. تمام های‌پرایم‌های pretrained backbone (مثل ViT، ResNet، HarDNet) و همچنین SAM روی این توزیع آموزش دیده‌اند. با استفاده از این آماره‌ها، تصاویر polyp به همان توزیعی منتقل می‌شوند که مدل انتظار دارد.

۲.۳ Pipeline آموزش (train_transforms)

```

1 train_transforms = Compose([
2     LoadImageD(KEYS),
3     EnsureChannelFirstD(keys=KEYS),
4     ScaleIntensityRangeD(
5         keys=[args['image_key']],
6         a_min=0.0, a_max=255.0,
7         b_min=0.0, b_max=1.0,
8         clip=True
9     ),
10    NormalizeIntensityD(
11        keys=[args['image_key']],
12        subtrahend=IMAGENET_MEAN,
13        divisor=IMAGENET_STD,
14        channel_wise=True
15    ),
16    ResizeWithRatio(
17        image_key=KEYS[0],
18        mask_key=KEYS[1],
19        image_size=sam_image_dim,
20        image_mode='bilinear',
21        mask_mode='nearest'
22    ),
23    ResizeWithPadOrCropd(
24        keys=KEYS,
25        spatial_size=(sam_image_dim, sam_image_dim),
26        mode='constant'
27    ),

```

```

۲۸ RandFlipD(keys=KEYS, prob=0.5, spatial_axis=0),
۲۹ RandFlipD(keys=KEYS, prob=0.5, spatial_axis=1),
۳۰ RandRotate90D(keys=KEYS, prob=0.5, spatial_axes=(0, 1)),
۳۱ EnsureTyped(keys=KEYS, track_meta=False)
۳۲ ])

```

تحلیل خط به خط:

جدول ۲: تحلیل خط به خط train_transforms

#	Transform	عملکرد
۱	LoadImageD	بارگذاری فایل‌های PNG از دیسک. خروجی: MetaTensor با شکل (H, W, C) در بازه‌ی [0, 255]
۲	EnsureChannelFirstD	اطمینان از اینکه بُعد کانال در ابتدا است. خروجی: (C, H, W)
۳	ScaleIntensityRangeD	نگاشت خطی $[0, 255] \rightarrow [0, 1]$ با تقسیم بر ۲۵۵. این تقسیم ساده است (نه min-max) تا با SAM و HardNet سازگار باشد.
۴	NormalizeIntensityD	نرمال‌سازی z-score با آماره‌های ImageNet. $x_{norm} = \frac{x - \mu_{ImageNet}}{\sigma_{ImageNet}}$ خروجی در بازه‌ی تقریبی [-2.12, +2.64].
۵	ResizeWithRatio	تغییر اندازه با حفظ نسبت ابعاد. مثلاً (529, 622) $\rightarrow$ (871, 1024).
۶	ResizeWithPadOrCropD	پد کردن به مربع (1024, 1024) با مقدار ثابت ۰. مقدار ۰ در فضای نرمال‌شده = خاکستری خنثی (میانگین ImageNet).
۷	RandFlipD (axis=0)	قرینه‌کردن عمودی با احتمال ۵۰٪.
۸	RandFlipD (axis=1)	قرینه‌کردن افقی با احتمال ۵۰٪.
۹	RandRotate90D	چرخش تصادفی ۹۰ درجه با احتمال ۵۰٪ (۴ زاویه ممکن: ۰°, ۹۰°, ۱۸۰°, ۲۷۰°).
۱۰	EnsureTyped	تبدیل به tensor ساده (حذف MetaTensor). track_meta=False برای صرفه‌جویی در حافظه و سرعت.

جریان ابعاد و بازه‌ی مقادیر:

۳.۳ Pipeline اعتبارسنجی/تست (val_test_transforms)

```

۱ val_test_transforms = Compose([
۲     LoadImageD(KEYS),
۳     EnsureChannelFirstD(keys=KEYS),
۴     ScaleIntensityRangeD(
۵         keys=[args['image_key']],
۶         a_min=0.0, a_max=255.0,
۷         b_min=0.0, b_max=1.0,
۸         clip=True
۹     ),
۱۰    NormalizeIntensityD(
۱۱        keys=[args['image_key']],
۱۲        subtrahend=IMAGENET_MEAN,
۱۳        divisor=IMAGENET_STD,
۱۴        channel_wise=True

```

جدول ۳: جریان ابعاد و بازه‌ی مقادیر در train_transforms

#	Transform	شکل	بازه
۰	PNG ورودی	(529, 622, 3)	[0, 255]
۱	LoadImageD	(529, 622, 3)	[0, 255]
۲	EnsureChannelFirstD	(3, 529, 622)	[0, 255]
۳	ScaleIntensityRangeD	(3, 529, 622)	[0, 1]
۴	NormalizeIntensityD	(3, 529, 622)	[-2.12, +2.64]
۵	ResizeWithRatio	(3, 871, 1024)	[-2.12, +2.64]
۶	ResizeWithPadOrCropd	(3, 1024, 1024)	[-2.12, +2.64]
۷-۹	Augmentations	(3, 1024, 1024)	[-2.12, +2.64]
۱۰	EnsureTyped	(3, 1024, 1024)	[-2.12, +2.64]

```

۱۵ ),
۱۶ ResizeWithRatio(
۱۷     image_key=KEYS[0],
۱۸     mask_key=KEYS[1],
۱۹     image_size=sam_image_dim,
۲۰     image_mode='bilinear',
۲۱     mask_mode='nearest'
۲۲ ),
۲۳ ResizeWithPadOrCropd(
۲۴     keys=KEYS,
۲۵     spatial_size=(sam_image_dim, sam_image_dim),
۲۶     mode='constant'
۲۷ ),
۲۸ EnsureTyped(keys=KEYS, track_meta=True)
۲۹ ])
```

تفاوت با train_transforms:

val_test	train	Transform
✓	✓	LoadImageD
✓	✓	EnsureChannelFirstD
✓	✓	ScaleIntensityRangeD
✓	✓	NormalizeIntensityD
✓	✓	ResizeWithRatio
✓	✓	ResizeWithPadOrCropd
×	✓	RandFlipD (axis=0)
×	✓	RandFlipD (axis=1)
×	✓	RandRotate90D
True	False	EnsureTyped (track_meta)

نکته‌ی مهم درباره‌ی track_meta:

- در train و val: track_meta=False برای صرفه‌جویی در حافظه.

- در test.py: track_meta=True چون test.py از imgs.meta['filename_or_obj'] برای استخراج نام فایل استفاده می‌کند.
- اگر track_meta=False باشد، متادیتا حذف می‌شود و test.py خطای AttributeError می‌دهد.

۴ تحلیل اثر هر Transform روی داده

۱.۴ گام ۱: LoadImageD

عملکرد: بارگذاری تصویر و ماسک از فایل‌های PNG روی دیسک.
ورودی: مسیر فایل (string): خروجی: MetaTensor در بازه [0, 255].

مثال عملی: برای یک تصویر polyp از Kvasir-SEG:

- فایل تصویر: cju0u82z3cuma0835wlxrnrv.png
- ابعاد اصلی: (H, W, C): (529, 622, 3)
- بازه: [0, 255]
- (MONAI float32 dtype: خودکار تبدیل می‌کند)

۲.۴ گام ۲: EnsureChannelFirstD

عملکرد: انتقال بُعد کانال به ابتدا (اگر در ابتدا نباشد).
تبدیل: (H, W, C) → (C, H, W)
مثال: (529, 622, 3) → (3, 529, 622)

۳.۴ گام ۳: ScaleIntensityRanged

عملکرد: نگاشت خطی از بازه [0, 255] به [0, 1].
فرمول:

$$x_{scaled} = \frac{x - a_{min}}{a_{max} - a_{min}} \times (b_{max} - b_{min}) + b_{min}$$

با مقادیر $b_{max}=1$, $b_{min}=0$, $a_{max}=255$, $a_{min}=0$:

$$x_{scaled} = \frac{x}{255}$$

مثال:

- پیکسل ۰ → ۰.۰
- پیکسل ۱۲۸ → ۰.۵
- پیکسل ۲۵۵ → ۰.۱

چرا ScaleIntensityRanged و نه ScaleIntensityD؟

- ScaleIntensityD(minv=0, maxv=1): از min-max استفاده می‌کند → هر تصویر بازه متفاوتی می‌گیرد.
- ScaleIntensityRanged(0, 255, 0, 1): تقسیم ساده بر ۲۵۵ → همه تصاویر یکسان.
- تقسیم بر ۲۵۵ با SAM و HardNet سازگار است (چون آن‌ها انتظار ورودی [0, 1] با تقسیم بر ۲۵۵ دارند).

۴.۴ گام ۴: NormalizeIntensityD

عملکرد: نرمال‌سازی z-score با آماره‌های ImageNet.
فرمول:

$$x_{norm} = \frac{x_{scaled} - \mu_{ImageNet}}{\sigma_{ImageNet}}$$

محاسبه برای هر کانال:

خروجی نهایی: بازه تقریبی [-2.12, +2.64]، میانگین ۰، انحراف معیار ۱.

کانال	mean	std	حداقل ($x=0$)	حداکثر ($x=1$)
R	۴۸۵.۰	۲۲۹.۰	$(0-0.485)/0.229 = -2.118$	$(1-0.485)/0.229 = +2.249$
G	۴۵۶.۰	۲۲۴.۰	$(0-0.456)/0.224 = -2.036$	$(1-0.456)/0.224 = +2.429$
B	۴۰۶.۰	۲۲۵.۰	$(0-0.406)/0.225 = -1.804$	$(1-0.406)/0.225 = +2.640$

سازگاری با **SAM**: کد رسمی SAM از $\text{pixel_mean} = [123.675, 116.28, 103.53]$ و $\text{pixel_std} = [58.395, 57.12, 57.375]$ استفاده می‌کند که دقیقاً برابر $\text{ImageNet_mean} \times 255$ و $\text{ImageNet_std} \times 255$ هستند. پس pipeline ما همان توزیع SAM را تولید می‌کند.

۵.۴ گام ۵: **ResizeWithRatio**

عملکرد: تغییر اندازه با حفظ نسبت ابعاد.
منطق: ضلع بزرگ‌تر به image_size (مثلاً ۱۰۲۴) می‌رسد، ضلع دیگر با همان نسبت مقیاس می‌شود.
مثال‌های عددی:

محاسبه	ورودی (H, W)	خروجی (H, W)
$871 = \text{round}(529 \times 1024/622)$	(529, 622)	(871, 1024)
$572 = \text{round}(1072 \times 1024/1920)$	(1920, 1072)	(1024, 572)
$768 = \text{round}(288 \times 1024/384)$	(384, 288)	(1024, 768)
مربع باقی می‌ماند	(512, 512)	(1024, 1024)

تفکیک: mode:

- **تصویر:** mode='bilinear' → درون‌یابی نرم.
- **ماسک:** mode='nearest' → حفظ مقادیر دودویی.

۶.۴ گام ۶: **ResizeWithPadOrCrop**

عملکرد: پد کردن به مربع (1024, 1024) با مقدار ثابت ۰.
مثال:

- $(3, 1024, 1024) \rightarrow (3, 871, 1024)$ با پد ۱۵۳ پیکسل در ارتفاع.
- پدینگ متقارن: ۷۶ پیکسل بالا + ۷۷ پیکسل پایین.

معنای مقدار ۰ در پدینگ:

- بعد از $\text{NormalizeIntensityD}$ ، ۰ = میانگین ImageNet = خاکستری خنثی.
- این با SAM سازگار است (SAM خودش هم بعد از pad normalize می‌کند).
- اگر پدینگ قبل از نرمال‌سازی بود، ۰ = سیاه مطلق و به 2.12- تبدیل می‌شد که یک لبه‌ی مصنوعی شدید ایجاد می‌کرد.

۷.۴ گام‌های ۷-۹: **Augmentation**

- **RandFlipD (axis=0):** قرینه‌کردن عمودی با احتمال ۰.۵.
- **RandFlipD (axis=1):** قرینه‌کردن افقی با احتمال ۰.۵.
- **RandRotate90D:** چرخش تصادفی ۹۰ درجه با احتمال ۰.۵ (۴ زاویه ممکن).
- **ترکیب:** چهار حالت با احتمال مساوی ۰.۲۵٪:

- بدون flip
- فقط افقی

- فقط عمودی
- هر دو

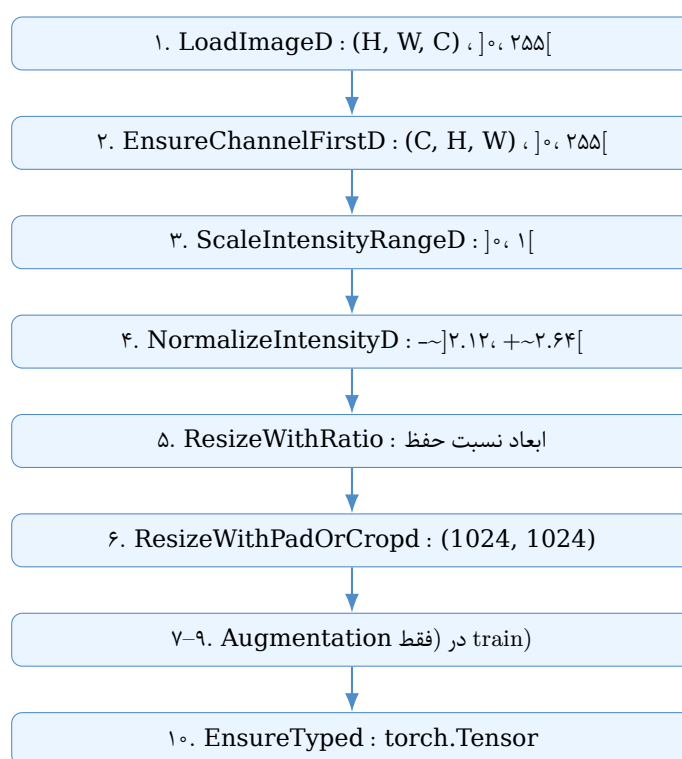
نکته‌ی مهم: RandFlipD و RandRotate90D روی هر دو تصویر و ماسک اعمال می‌شوند (چون keys=KEYS). این برای حفظ هم‌ترازی تصویر و ماسک ضروری است.

۸.۴ گام ۱۰: EnsureTyped

عملکرد: اطمینان از tensor بودن داده و تعیین track_meta تفاوت:

- train/val: track_meta=False → خروجی torch.Tensor ساده.
- test: track_meta=True → خروجی MetaTensor با متادیتا.

۵ جریان کامل داده – نمودار



شکل ۲: جریان کامل داده در train_transforms

۶ نسخه‌های بهبوددهنده (برای آینده)

کد اصلی شامل کامنت‌هایی است که بهبودهای آینده را پیشنهاد می‌دهد. این بخش، آن‌ها را با جزئیات توضیح می‌دهد.

۱.۶ بهبود ۱: RandRotated (جایگزین RandRotate90D)

```

۱ RandRotated(
۲     keys=KEYS,
۳     range_x=30 * math.pi / 180, # ±30 درجه‌های راندی
۴     prob=0.5,
۵     keep_size=True
۶ )
  
```

مزیت: چرخش پیوسته در بازه $[-30^\circ, +30^\circ]$ به جای فقط ۴ زاویه‌ی گسسته. **دلیل:** polyp در هر زاویه‌ای ممکن است ظاهر شود.

۲.۶ بهبود ۲: RandZoomD

```
1 RandZoomD(  
2     keys=KEYS,  
3     min_zoom=0.8, max_zoom=1.20,  
4     mode=('bilinear', 'nearest'),  
5     prob=0.5,  
6     padding_mode='constant',  
7     align_corners=False  
8 )
```

محل پیشنهادی: بعد از ResizeWithPadOrCropd. مزیت: تغییر مقیاس polyp (شبیه‌سازی تغییر فاصله‌ی دوربین).

۳.۶ بهبود ۳: RandAdjustContrastD

```
1 RandAdjustContrastD(  
2     keys=[args['image_key']],  
3     prob=0.3,  
4     gamma=(0.7, 1.5)  
5 )
```

محل پیشنهادی: قبل از ScaleIntensityRangeD (روی [0, 255]). مزیت: شبیه‌سازی تغییرات کنتراست.

۴.۶ بهبود ۴: RandGaussianNoiseD

```
1 RandGaussianNoiseD(  
2     keys=[args['image_key']],  
3     prob=0.2,  
4     mean=0.0,  
5     std=0.05  
6 )
```

محل پیشنهادی: قبل از ScaleIntensityRangeD. مزیت: مقاوم‌سازی به نویز سنسور.

۵.۶ بهبود ۵: RandShiftIntensityD

```
1 RandShiftIntensityD(  
2     keys=[args['image_key']],  
3     offsets=0.1,  
4     prob=0.5  
5 )
```

محل پیشنهادی: بین ScaleIntensityRangeD و NormalizeIntensityD (روی مقادیر [0, 1]). مزیت: شبیه‌سازی تغییرات روشنایی. نکته: offset=0.1 فقط در بازه‌ی [0, 1] معنای $\pm 10\%$ دارد.

۶.۶ بهبود ۶: RandGaussianSmoothD

```
1 RandGaussianSmoothD(  
2     keys=[args['image_key']],  
3     prob=0.1,  
4     sigma_x=(0.5, 1.0),  
5     sigma_y=(0.5, 1.0)  
6 )
```

محل پیشنهادی: قبل از ScaleIntensityRangeD. مزیت: شبیه‌سازی عدم فوکوس دوربین.

۷ جمع‌بندی

نکات کلیدی

۱. ترتیب حیاتی است:

جدول ۴: خلاصه‌ی نهایی جریان داده

#	Transform	شکل	بازه
۰	ورودی PNG	(H, W, 3)	[0, 255]
۱	LoadImageD	(H, W, 3)	[0, 255]
۲	EnsureChannelFirstD	(3, H, W)	[0, 255]
۳	ScaleIntensityRangeD	(3, H, W)	[0, 1]
۴	NormalizeIntensityD	(3, H, W)	[-2.12, +2.64]
۵	ResizeWithRatio	(3, h', w')	[-2.12, +2.64]
۶	ResizeWithPadOrCropd	(3, 1024, 1024)	[-2.12, +2.64]
۷-۹	(train) Augmentations	(3, 1024, 1024)	[-2.12, +2.64]
۱۰	EnsureTyped	(3, 1024, 1024)	[-2.12, +2.64]

- Scale و Normalize قبل از Resize و Pad.
- اگر برعکس شود، پدینگ 0 در فضای اصلی = -2.12 در فضای نرمال شده → لبه‌ی مصنوعی مضر.
- ۲. تفکیک mode ضروری است:
 - bilinear برای تصویر (نرمی).
 - nearest برای ماسک (حفظ binary).
- ۳. **track_meta** برای **test = True**:
 - چون test.py از filename_or_obj استفاده می‌کند.
- ۴. Augmentations فقط روی تصویر و ماسک:
 - keys=KEYS برای RandRotate90D و RandFlipD.
 - keys=[image_key] برای augmentations رنگ (در بهبودهای آینده).
- ۵. هم‌راستایی با SAM و HarDNet:
 - ImageNet_std و ImageNet_mean = توزیع مورد انتظار های pretrained. backbone
 - ScaleIntensityRangeD(0, 255, 0, 1) = تقسیم ساده بر ۲۵۵.
 - نتیجه: توزیع نهایی mean=0, std=1 که SAM و HarDNet انتظار دارند.

پایپلاین فعلی = پایپلاین استاندارد: polyp این ترتیب transform ها دقیقاً همان چیزی است که مقالات PraNet، Polyp-، SANet، PVT استفاده می‌کنند، با تفاوت‌های کوچک (مثل RandRotate90D به جای RandRotated). پس نتایج شما با این baseline قابل مقایسه با مقالات معتبر است.